



**Université Mohammed
Premier
Faculté des Sciences
d'Oujda
Département
d'Informatique**



**Rapport de
Mini-Projet de Data Mining
Master Spécialisé Ingénierie Informatique M2I**

**Implémentation de l'arbre de décision ID3
pour la classification de données catégorielles
Étude de cas : jeu de données Nursery (UCI)**

Présenté par :

GHAYOUT Amine

BOUBECHRIE Rida

Encadré par :

Pr. Azzeddine Mazroui

Devant le jury :

1. Pr. Azzeddine Mazroui

Année universitaire : 2025-2026

Remerciements

Ce travail n'aurait pas pu voir le jour sans le soutien et les conseils de plusieurs personnes à qui nous souhaitons adresser nos sincères remerciements.

Nous remercions en premier lieu notre encadrant, le Pr. Azzeddine Mazroui, dont les cours sur les arbres de décision nous ont fourni des bases solides pour aborder ce projet. Ses explications claires sur l'entropie et l'algorithme ID3, ainsi que sa disponibilité pour répondre à nos questions, ont été précieuses tout au long de cette réalisation.

Nos remerciements s'adressent également à l'ensemble des enseignants et du personnel administratif de la Faculté des Sciences d'Oujda et du Master M2I, pour la qualité de l'encadrement pédagogique dont nous bénéficions à l'Université Mohammed Premier.

Enfin, nous remercions nos familles pour leur patience et leur soutien, ainsi que nos camarades de promotion pour les échanges que nous avons eus au fil de ce projet.

Résumé

Ce mini-projet porte sur l'implémentation, en Java et sans bibliothèque externe, de l'algorithme d'induction d'arbres de décision ID3[1]. Le travail s'appuie sur le jeu de données Nursery de l'UCI Machine Learning Repository, qui contient 12 960 candidatures d'inscription en crèche décrites par huit attributs catégoriels et réparties en cinq classes. Toutes les valeurs d'attributs sont des chaînes de caractères, ce qui correspond exactement au cadre du sujet n° 4 qui nous a été attribué.

L'algorithme ID3 est construit à partir des formules d'entropie de Shannon et de gain d'information présentées en cours. Une application web Spring Boot [2] / Thymeleaf permet de charger les données, de lancer l'apprentissage, de prédire la classe d'une nouvelle candidature en affichant le chemin de décision suivi, et d'évaluer les performances du modèle sur un jeu de test indépendant.

Sur les 2 592 instances de test (20 % du corpus), l'exactitude globale atteint 98,30 %. Les métriques macro sont plus modestes ($F1 \approx 75\%$) en raison d'une classe quasiment absente du dataset, ce qui est une limite inhérente aux données et non à l'algorithme.

Mots-clés — arbre de décision, ID3, entropie, gain d'information, données catégorielles, Nursery, Spring Boot.

Abstract

This project covers the from-scratch Java implementation of the ID3 decision tree induction algorithm, applied to the UCI Nursery dataset (12,960 instances, eight categorical attributes, five classes). All attribute values are character strings, which matches exactly the scope of the assigned topic.

The ID3 engine relies on Shannon entropy and information gain as presented in the Data Mining course. A Spring Boot / Thymeleaf web application handles data loading, tree construction, interactive prediction with decision path display, and model evaluation on a held-out test set.

On 2,592 test instances (20% of the corpus), overall accuracy reaches 98.30%. Macro-averaged metrics are lower (~75% F1) due to one near-absent class in the dataset — a data limitation, not an algorithmic one.

Keywords — decision tree, ID3, entropy, information gain, categorical data, Nursery, Spring Boot.

Sommaire

Remerciements	3
Résumé	4
Abstract.....	5
Sommaire.....	6
Liste des figures.....	8
Liste des tableaux	9
Introduction générale.....	10
Contexte.....	10
Problématique	10
Objectifs.....	10
Organisation du rapport	11
Chapitre 1 — Présentation du sujet, du jeu de données et de la méthode ID3.....	12
1.1 Cadre du mini-projet.....	12
1.2 Jeu de données Nursery (UCI).....	12
1.3 Rappel théorique — Entropie, gain d'information et algorithme ID3	13
1.4 Synthèse	14
Chapitre 2 — Conception du système	15
2.1 Pipeline de traitement des données et architecture logicielle	15
2.2 Modélisation UML	16
2.3 Synthèse	18
Chapitre 3 — Environnement technique et implémentation de l'algorithme ID3	19
3.1 Environnement et outils utilisés.....	19
3.2 Implémentation de l'algorithme ID3	19
3.3 Encodage des attributs catégoriels.....	21
3.4 Synthèse	21
Chapitre 4 — Interface graphique de l'application.....	22
4.1 Page d'accueil.....	22
4.2 Page Apprentissage.....	22
4.3 Page Prédiction	24
4.4 Page Évaluation	25

4.5 Export de l'arbre de décision.....	26
4.6 Synthèse	27
Chapitre 5 — Protocole de test, résultats et discussion.....	28
5.1 Protocole expérimental	28
5.2 Résultats globaux.....	28
5.3 Résultats détaillés par classe.....	29
5.4 Discussion.....	29
5.5 Synthèse.....	30
Conclusion générale	31
Synthèse du travail réalisé	31
Difficultés rencontrées.....	31
Perspectives d'amélioration	31
Bibliographie	32

Liste des figures

- Figure 1.1 — Répartition des classes du jeu de données Nursery
- Figure 2.1 — Pipeline de traitement des données
- Figure 2.2 — Architecture logicielle en couches
- Figure 2.3 — Diagramme de cas d'utilisation
- Figure 2.4 — Diagramme de classes simplifié
- Figure 2.5 — Diagramme de séquence — module de prédiction
- Figure 3.1 — Organigramme de l'algorithme ID3
- Figure 4.1 — Page d'accueil (haut)
- Figure 4.2 — Page d'accueil — jeu de données et statistiques
- Figure 4.3 — Page Apprentissage — description des données
- Figure 4.4 — Page Apprentissage — chargement et construction de l'arbre
- Figure 4.5 — Aperçu interactif de l'arbre ID3
- Figure 4.6 — Formulaire de prédiction
- Figure 4.7 — Résultat d'une prédiction
- Figure 4.8 — Chemin de décision suivi
- Figure 4.9 — Page Évaluation — métriques globales et par classe
- Figure 4.10 — Matrice de confusion
- Figure 4.11 — Export HTML de l'arbre complet

Liste des tableaux

Tableau 1.1 — Répartition des classes dans le corpus Nursery

Tableau 1.2 — Attributs et modalités du jeu de données Nursery

Tableau 2.1 — Correspondance couches logicielles — classes Java

Tableau 3.1 — Outils et technologies utilisés

Tableau 3.2 — Paramètres et critères d'arrêt de l'algorithme ID3

Tableau 5.1 — Répartition apprentissage / test

Tableau 5.2 — Résultats globaux sur le jeu de test

Tableau 5.3 — Métriques de performance par classe

Tableau 5.4 — Matrice de confusion — jeu de test

Introduction générale

Contexte

La fouille de données regroupe des techniques permettant d'extraire des connaissances utiles à partir de grandes quantités de données. Parmi elles, la classification supervisée est l'une des plus répandues : à partir d'exemples déjà étiquetés, on construit un modèle capable d'attribuer une classe à de nouveaux individus. Les arbres de décision constituent une famille particulièrement appréciée de ce type de modèles, car leurs décisions sont faciles à lire et à expliquer — chaque prédiction se réduit à une suite de tests simples sur les attributs descriptifs.

Dans le cadre du module Data Mining du Master M2I, le chapitre 5 du cours couvre les fondements théoriques [3] de cette famille de méthodes : entropie de Shannon, gain d'information et algorithme ID3 (Iterative Dichotomiser 3). À la suite de ce chapitre, chaque binôme a reçu un sujet de mini-projet à réaliser.

Problématique

Notre binôme s'est vu attribuer le sujet n° 4 : « Implémentation de l'arbre de décision pour les chaînes de caractères ». L'intitulé pointe directement vers le cas où tous les attributs prédictifs prennent leurs valeurs dans un ensemble fini de modalités textuelles par exemple « ensoleillé », « couvert » ou « pluvieux » pour un attribut météo. C'est précisément le cadre de l'algorithme ID3 tel qu'il est présenté en cours.

La question pratique que nous devons résoudre est double : d'un côté, concevoir et coder en Java l'algorithme ID3 sans recourir à une bibliothèque de machine learning ; de l'autre, l'intégrer dans une application dotée d'une interface graphique, capable de charger des données, d'apprendre l'arbre, de classer de nouveaux individus et d'évaluer objectivement les performances du modèle. Pour l'étude de cas, nous avons retenu le jeu de données Nursery (UCI), dont les huit attributs et la classe cible sont tous des variables qualitatives codées sous forme de chaînes de caractères — ce qui correspond exactement à l'énoncé du sujet.

Objectifs

Ce projet vise concrètement à :

- Préparer le jeu de données Nursery : comprendre les attributs et les classes, puis diviser le corpus en 80 % pour l'apprentissage et 20 % pour le test.
- Implémenter l'algorithme ID3 en Java (entropie, gain d'information, construction récursive de l'arbre, critères d'arrêt) sans bibliothèque externe.
- Développer une interface web complète (Spring Boot, Thymeleaf) couvrant le chargement des données, la construction de l'arbre, la prédiction interactive avec affichage du chemin de décision, et l'évaluation du modèle.
- Évaluer le modèle sur le jeu de test (exactitude, précision, rappel, F1, matrice de confusion) et discuter les résultats.
- Rédiger un rapport complet conforme aux recommandations du module.

Organisation du rapport

Le rapport est structuré en cinq chapitres. Le premier présente le contexte du sujet, le jeu de données Nursery et les notions théoriques d'entropie et d'ID3. Le deuxième décrit la conception du système (architecture, diagrammes UML). Le troisième détaille l'implémentation technique. Le quatrième fait le tour de l'interface graphique. Le cinquième présente le protocole de test et discute les résultats. Le rapport se conclut par un bilan du travail réalisé et des perspectives d'amélioration.

Chapitre 1 — Présentation du sujet, du jeu de données et de la méthode ID3

1.1 Cadre du mini-projet

Pour l'année universitaire 2025/2026, le module Data Mining du Master M2I a proposé huit sujets de mini-projets, couvrant des techniques variées : k-moyennes, Naïve Bayes, règles d'association, arbres de décision, réseaux de neurones, classification hiérarchique, k plus proches voisins, et segmentation par voisinage dense. Chaque binôme ou trinôme devait choisir un sujet, l'implémenter et remettre un rapport et les programmes au plus tard le 21 juin 2026.

Le sujet n° 4, « Implémentation de l'arbre de décision pour les chaînes de caractères », a été confié à notre binôme — GHAYOUT Amine et BOUBECHRIE Rida — sous la supervision du Pr. Azzeddine Mazroui. Les conditions imposées par le module (taux de plagiat inférieur à 15 %, interface graphique obligatoire) ont guidé nos choix techniques depuis le départ.

1.2 Jeu de données Nursery (UCI)

Le jeu de données Nursery, disponible sur l'UCI Machine Learning Repository[4], a été créé pour évaluer des demandes d'inscription d'enfants dans des crèches en ex-Yougoslavie. Chaque ligne représente une candidature, décrite par huit attributs qualitatifs, et associée à l'une des cinq classes de recommandation possibles. Ce dataset nous a semblé idéal pour le sujet : tous ses attributs, y compris la classe cible, sont des chaînes de caractères, sans aucune valeur numérique continue.

Le dossier « donnees/ » du projet contient trois fichiers : nursery.data (données brutes UCI, 12 960 lignes), entraînement.csv (~10 368 instances, 80 %) et test.csv (~2 592 instances, 20 %). Le découpage a été réalisé par le script Python « preparer_nursery.py », avec une graine aléatoire fixée à 42 pour garantir la reproductibilité des résultats.

Classe	Effectif	Pourcentage
not_recom	4 320	33,33 %
priority	4 266	32,92 %
spec_prior	4 044	31,20 %
very_recom	328	2,53 %
recommend	2	0,02 %
Total	12 960	100,00 %

Tableau 1.1 — Répartition des classes dans le corpus Nursery

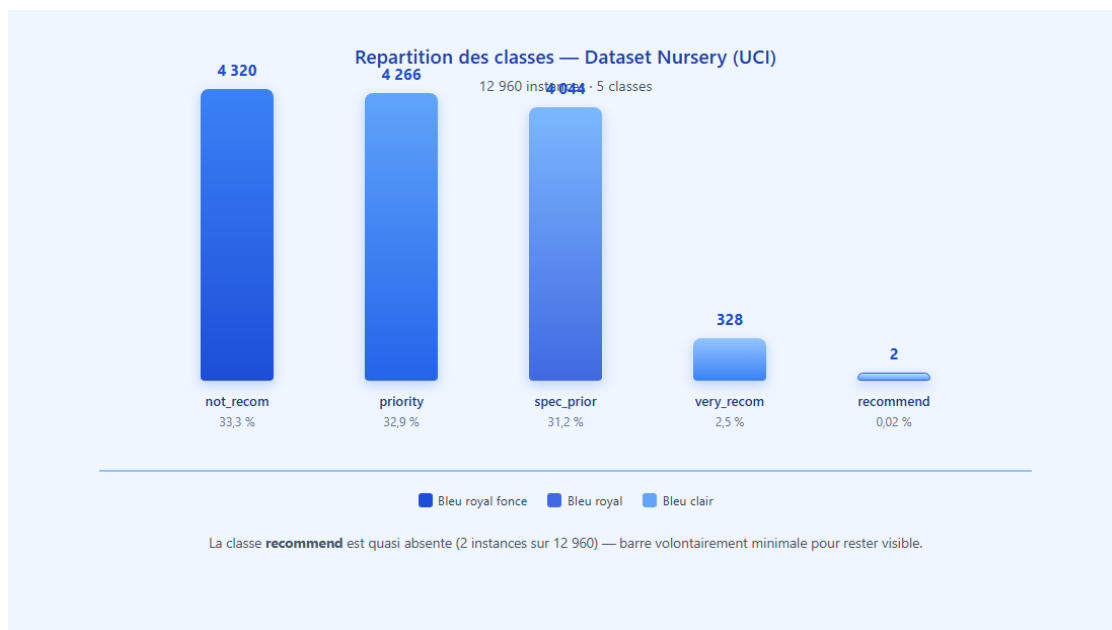


Figure 1.1 — Répartition des classes du jeu de données Nursery

On remarque immédiatement que les classes `not_recom`, `priority` et `spec_prior` représentent à elles trois plus de 97 % du corpus, tandis que `very_recom` ne couvre que 2,5 % des instances et que `recommend` est quasi absente (2 instances sur 12 960). Ce déséquilibre marqué aura une influence directe sur les métriques d'évaluation discutées au chapitre 5.

Attribut	Modalités
parents	usual, pretentious, great_pret
has_nurs	proper, less_proper, improper, critical, very_crit
form	complete, completed, incomplete, foster
children	1, 2, 3, more
housing	convenient, less_conv, critical
finance	convenient, inconv
social	nonprob, slightly_prob, problematic
health	recommended, priority, not_recom

Tableau 1.2 — Attributs et modalités du jeu de données Nursery

1.3 Rappel théorique — Entropie, gain d'information et algorithme ID3

L'algorithme ID3, proposé par Quinlan en 1986, construit un arbre de décision de façon récursive et descendante. À chaque nœud, il choisit l'attribut qui sépare le mieux les exemples par rapport à leur classe, en se basant sur la notion d'entropie de Shannon[5].

Pour un ensemble S d'exemples répartis en k classes, avec p_i la proportion d'exemples de la classe i , l'entropie de S mesure le degré d'hétérogénéité :

$$E(S) = - \sum_i p_i \times \log_2(p_i)$$

Cette valeur est nulle quand tous les exemples appartiennent à une même classe (ensemble pur), et maximale quand les classes sont équiréparties. Pour choisir l'attribut à tester, ID3 calcule le gain d'information de chaque attribut A disponible. En notant S_v le sous-ensemble des exemples pour lesquels A prend la valeur v :

$$Gain(S, A) = E(S) - \sum_v (|S_v| / |S|) \times E(S_v)$$

L'attribut qui maximise ce gain est retenu. Un nœud interne lui est associé, et la procédure est appelée récursivement sur chaque sous-ensemble. La récursion s'arrête quand l'ensemble courant est pur, quand il n'y a plus d'attribut disponible, ou quand un critère d'arrêt (profondeur maximale, effectif minimal) est atteint. Dans ce cas, le nœud courant devient une feuille étiquetée par la classe majoritaire du sous-ensemble.

1.4 Synthèse

Ce chapitre a posé les bases du projet : le sujet porte bien sur un arbre de décision ID3 appliqué à des données entièrement catégorielles, le jeu de données Nursery répond parfaitement à cette contrainte, et les notions d'entropie et de gain d'information sont directement celles du cours. La suite du rapport décrit comment ces éléments ont été traduits en une application fonctionnelle.

Chapitre 2 — Conception du système

2.1 Pipeline de traitement des données et architecture logicielle

Avant de coder quoi que ce soit, nous avons défini le pipeline complet de traitement (figure 2.1) : depuis le fichier brut nursery.data jusqu'aux résultats de prédiction et d'évaluation, en passant par le découpage du corpus, l'encodage discret des modalités et la construction de l'arbre.

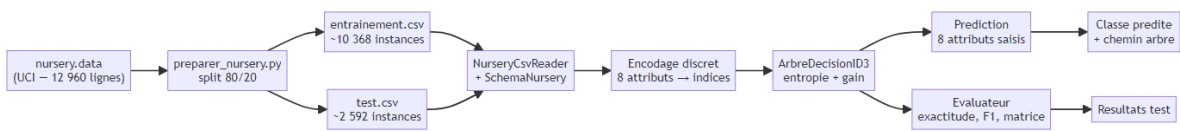


Figure 2.1 — Pipeline de traitement des données

Sur le plan architectural, l'application suit le patron MVC proposé par Spring Boot, organisé en quatre couches (figure 2.2) : présentation (contrôleur et templates Thymeleaf), services métier, accès aux données, et moteur ID3. Cette séparation rend le code lisible et facilite l'ajout éventuel d'un autre algorithme ou d'un autre jeu de données.

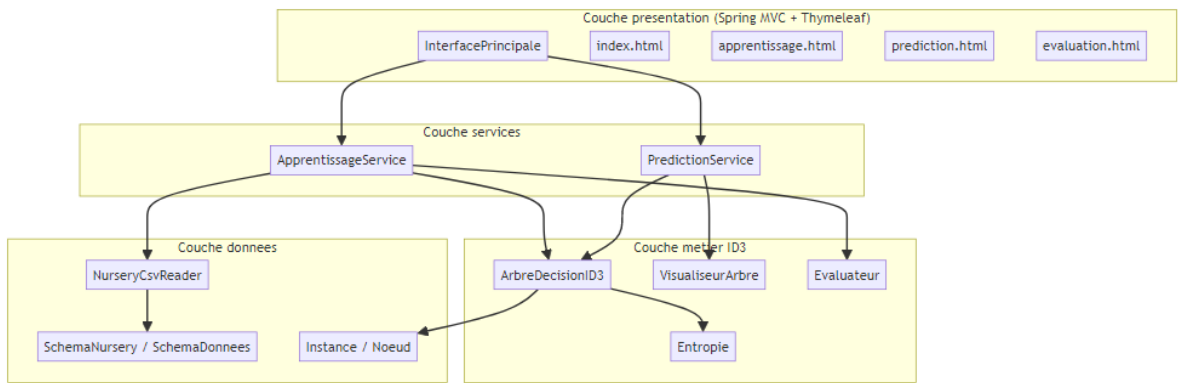


Figure 2.2 — Architecture logicielle en couches

Couche	Rôle	Principales classes
Présentation	Réception HTTP, sélection de la vue	InterfacePrincipale, index.html, apprentissage.html, prediction.html, evaluation.html
Services	Orchestration apprentissage / prédiction	ApprentissageService, PredictionService
Données	Lecture CSV, encodage des modalités	NurseryCsvReader, SchemaNursery, SchemaDonnees, Instance

Couche	Rôle	Principales classes
Métier ID3	Entropie, gain, arbre, évaluation	ArbreDecisionID3, Entropie, Noeud, VisualiseurArbre, Evalueur

Tableau 2.1 — Correspondance couches logicielles — classes Java

2.2 Modélisation UML

La figure 2.3 montre le diagramme de cas d'utilisation. L'acteur principal — l'utilisateur du binôme ou le jury lors de la démonstration — peut déclencher cinq actions : charger les données, construire l'arbre, soumettre une candidature, évaluer le modèle, et exporter l'arbre en HTML.

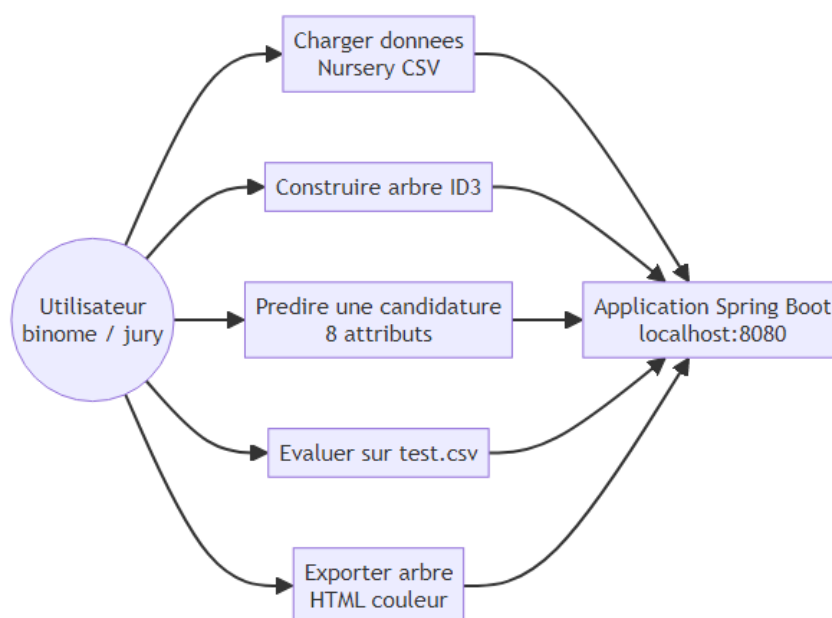


Figure 2.3 — Diagramme de cas d'utilisation

Le diagramme de classes (figure 2.4) montre les relations entre les principales classes Java. Le contrôleur délègue aux deux services, qui partagent `ArbreDecisionID3`. Cette classe manipule des objets `Noeud` (structure récursive), s'appuie sur `Entropie` pour les calculs et sur `Evalueur` pour les métriques.

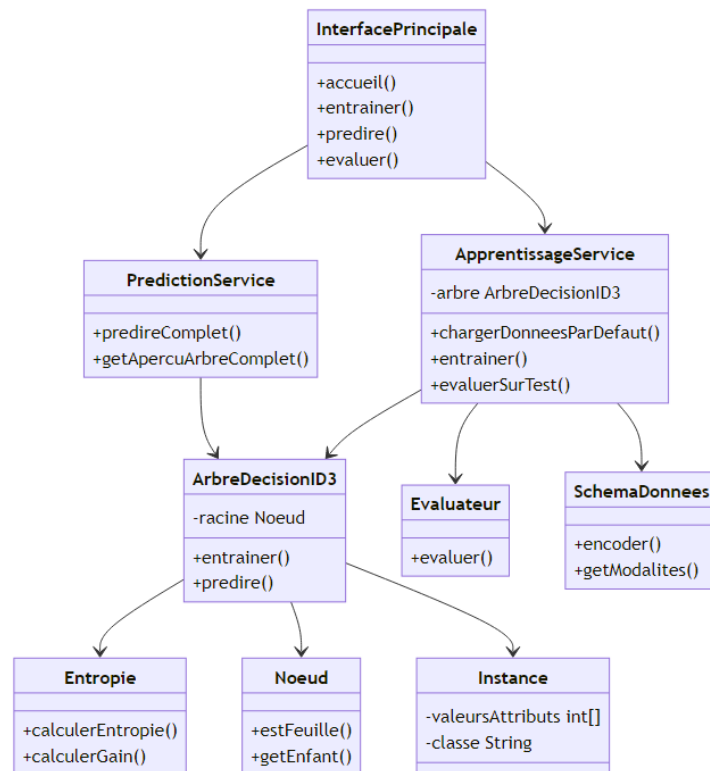


Figure 2.4 — Diagramme de classes simplifié

Enfin, la figure 2.5 détaille le déroulement d'une prédiction : l'utilisateur saisit ses huit attributs, le contrôleur passe la main au PredictionService, qui encode l'instance, interroge ArbreDecisionID3 pour obtenir la classe, puis VisualiseurArbre pour extraire le chemin de décision. Tout est renvoyé à l'utilisateur sous forme d'un mini-arbre visuel.

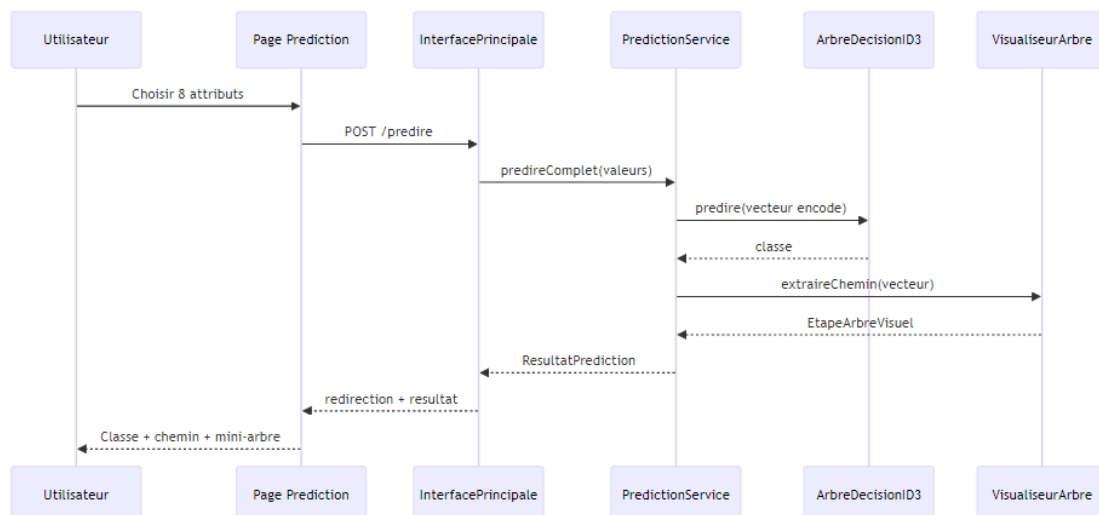


Figure 2.5 — Diagramme de séquence — module de prédiction

2.3 Synthèse

La conception retenue sépare clairement le moteur ID3 des couches applicatives et de l'interface. Cette organisation facilite la lecture du code, son évolution éventuelle, et la démonstration lors de la présentation du projet.

Chapitre 3 — Environnement technique et implémentation de l'algorithme ID3

3.1 Environnement et outils utilisés

Outil / technologie	Rôle dans le projet
Java 17	Langage principal — implémentation ID3 from scratch, sans bibliothèque de machine learning
Spring Boot 3.2.5	Serveur web embarqué, injection de dépendances, contrôleur MVC
Thymeleaf	Moteur de templates pour les pages HTML
Maven	Gestion des dépendances et compilation (pom.xml)
Python 3	Script <code>preparer_nursery.py</code> — découpage 80/20 du corpus Nursery
Mermaid [6]	Génération des diagrammes du rapport (architecture, UML, algorithme)
IntelliJ IDEA	Environnement de développement intégré utilisé pour le projet

Tableau 3.1 — Outils et technologies utilisés

Le projet est un projet Maven standard (`src/main/java`, `src/main/resources`). Le fichier `pom.xml` déclare deux dépendances principales : `spring-boot-starter-web` pour le serveur et `spring-boot-starter-thymeleaf` pour les vues. Le code Java est organisé par paquets fonctionnels, ce qui correspond aux couches décrites au chapitre 2.

3.2 Implémentation de l'algorithme ID3

La classe `Entropie` traduit directement les formules du cours. Sa méthode `calculerEntropie` parcourt les indices d'un sous-ensemble, compte les occurrences de chaque classe et calcule $-\sum p_i \log_2(p_i)$. Sa méthode `calculerGain` s'en sert pour évaluer l'apport de chaque attribut : elle calcule d'abord l'entropie initiale du sous-ensemble, partitionne les instances par modalité de l'attribut testé, recalcule l'entropie de chaque partition, et retourne la différence.

La classe `ArbreDecisionID3` orchestre la construction récursive. Sa méthode `construireArbre` reçoit la liste des indices d'instances du nœud courant, la liste des attributs encore disponibles et la profondeur. Elle détermine la classe majoritaire, vérifie si l'ensemble est pur ou si un critère d'arrêt est atteint, puis sélectionne l'attribut de gain maximal, crée un nœud interne, partitionne les instances et se rappelle récursivement sur chaque sous-ensemble.

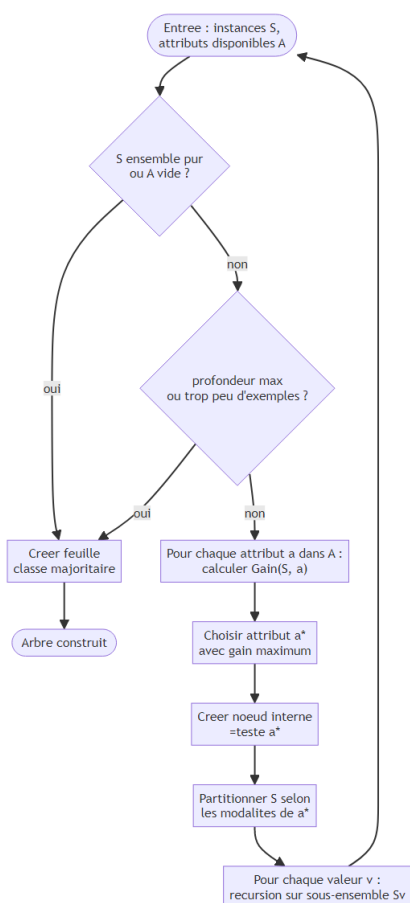


Figure 3.1 — Organigramme de l'algorithme ID3

Paramètre / critère	Valeur	Effet si atteint
Profondeur maximale	10 niveaux	Feuille (classe majoritaire)
Effectif minimal par feuille	2 instances	Feuille (classe majoritaire)
Ensemble pur	Toutes les instances ont la même classe	Feuille (cette classe)
Plus d'attributs disponibles	—	Feuille (classe majoritaire)
Gain d'information nul	$\text{Gain}(S, A) \leq 0$ pour tout A	Feuille (classe majoritaire)

Tableau 3.2 — Paramètres et critères d'arrêt de l'algorithme ID3

Sur le jeu de données Nursery, qui ne compte que huit attributs, la profondeur maximale de dix niveaux n'est jamais atteinte — elle joue le rôle de garde-fou pour d'éventuels autres jeux de données.

3.3 Encodage des attributs catégoriels

Pour manipuler les chaînes de caractères de façon efficace, la classe `SchemaNursery` définit, pour chacun des huit attributs, la liste ordonnée de ses modalités. La classe `SchemaDonnees` construit à partir de ces listes une table de correspondance modalité $\rightarrow$ indice entier. `NurseryCsvReader` lit les fichiers CSV et transforme chaque instance en un vecteur d'entiers (`int[]`), ce qui permet à l'algorithme ID3 de comparer des valeurs par simple égalité d'entiers plutôt que par comparaison de chaînes, et de calculer rapidement les effectifs nécessaires au calcul de l'entropie. La même table est utilisée en sens inverse pour afficher les résultats en clair dans l'interface.

3.4 Synthèse

Les classes `Entropie` et `ArbreDecisionID3` forment le cœur du projet. Leur implémentation est directement issue des formules et du pseudo-code du cours, sans aucune dépendance externe. L'encodage discret des modalités assure à la fois l'efficacité des calculs et la compatibilité avec n'importe quel jeu de données catégoriel.

Chapitre 4 — Interface graphique de l'application

4.1 Page d'accueil

La page d'accueil (figures 4.1 et 4.2) est la première chose que voit l'utilisateur quand il ouvre l'application. Elle présente le contexte académique du projet, résume les caractéristiques du jeu de données Nursery (nombre de classes, effectifs, attributs) et propose un parcours guidé en trois étapes : Apprentissage, Prédiction, Évaluation.

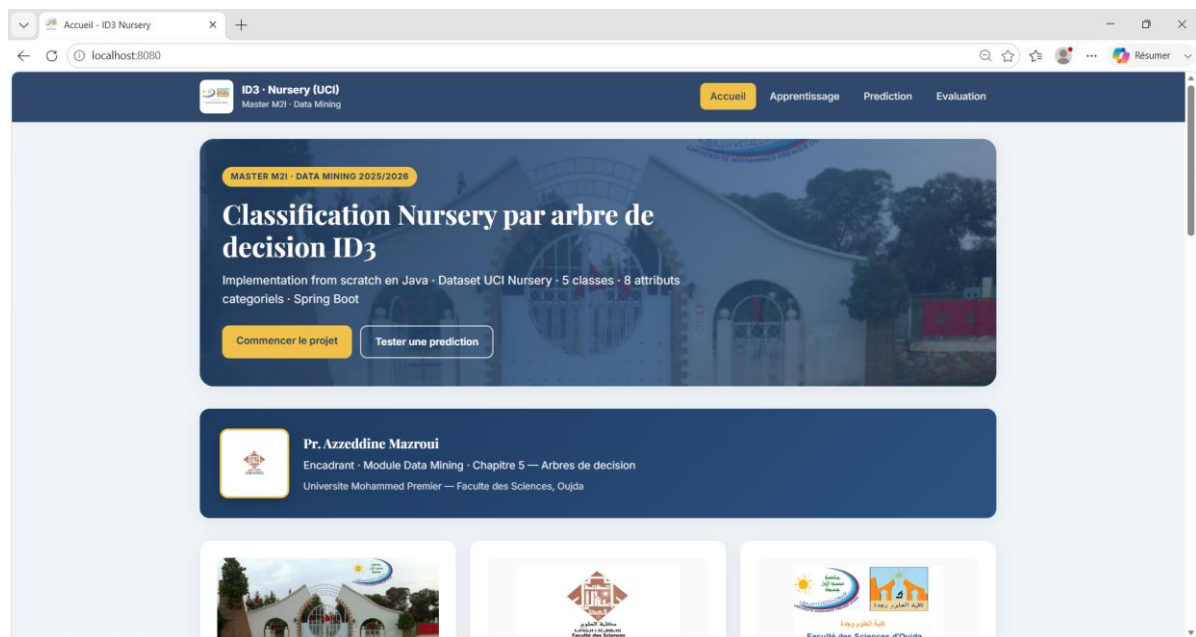


Figure 4.1 — Page d'accueil (haut)

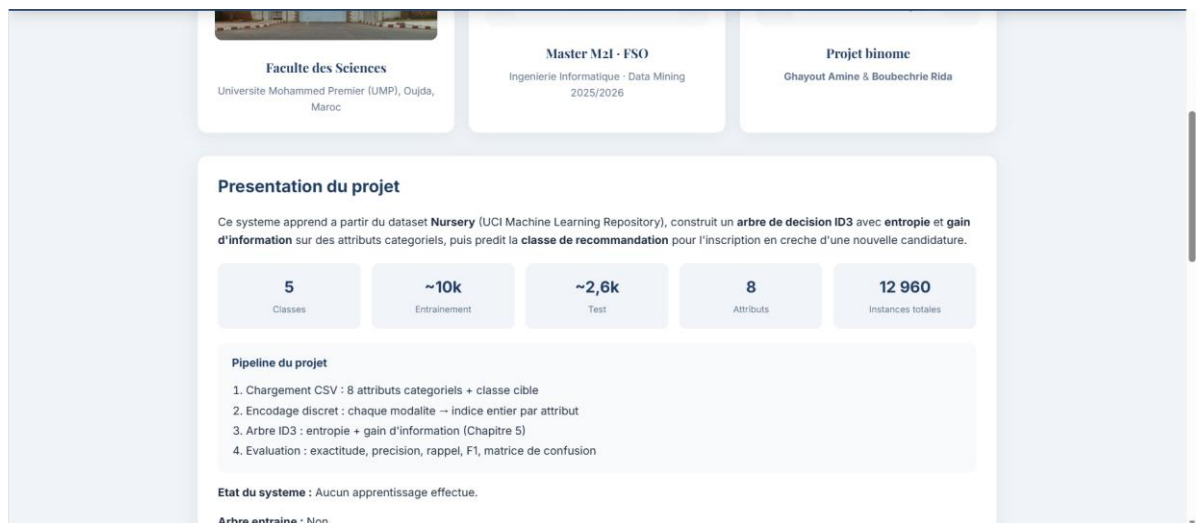


Figure 4.2 — Page d'accueil — jeu de données et statistiques

4.2 Page Apprentissage

La page Apprentissage couvre les deux premières étapes du pipeline : chargement des données et construction de l'arbre. Elle affiche d'abord le tableau des attributs et de leurs modalités (figure 4.3), puis l'état courant du système.

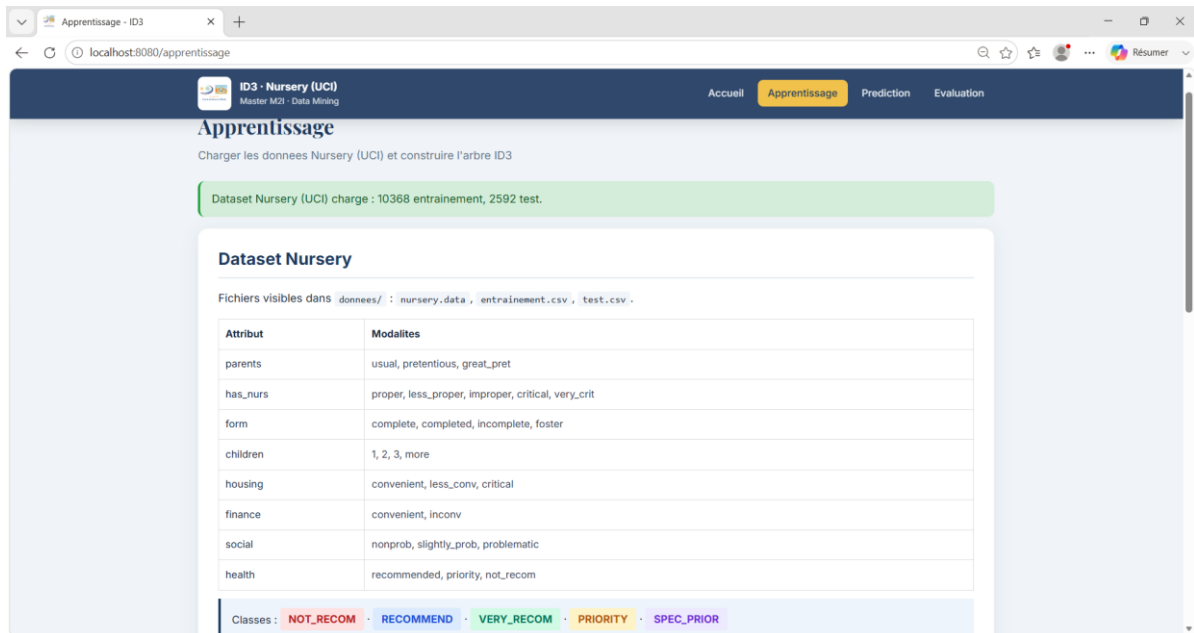


Figure 4.3 — Page Apprentissage — description des données

Deux boutons déclenchent les opérations : « Charger les données par défaut » lit directement les fichiers CSV intégrés au projet, tandis que « Construire l'arbre » lance l'algorithme ID3 sur les instances d'entraînement. Un chargement manuel de fichiers CSV externes est également proposé (figure 4.4).

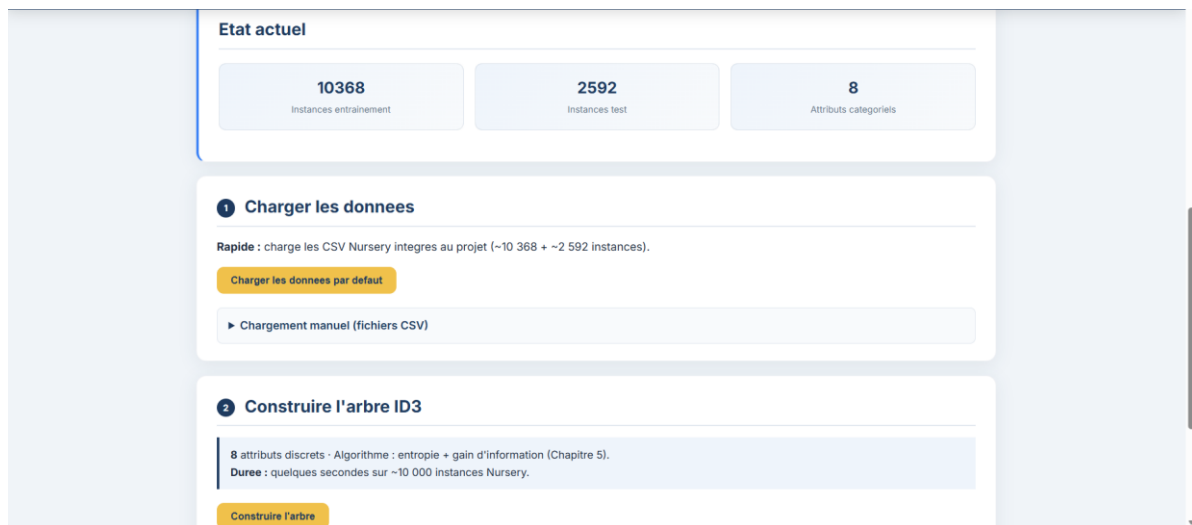


Figure 4.4 — Page Apprentissage — chargement et construction de l'arbre

Une fois l'arbre construit, un aperçu interactif s'affiche directement dans la page (figure 4.5) : les nœuds gris représentent les attributs testés, les branches rouges les modalités, et les feuilles colorées les classes prédites. Un contrôle de zoom et le défilement horizontal permettent d'explorer l'ensemble de la structure. Un lien permet aussi de télécharger une version HTML autonome de l'arbre.

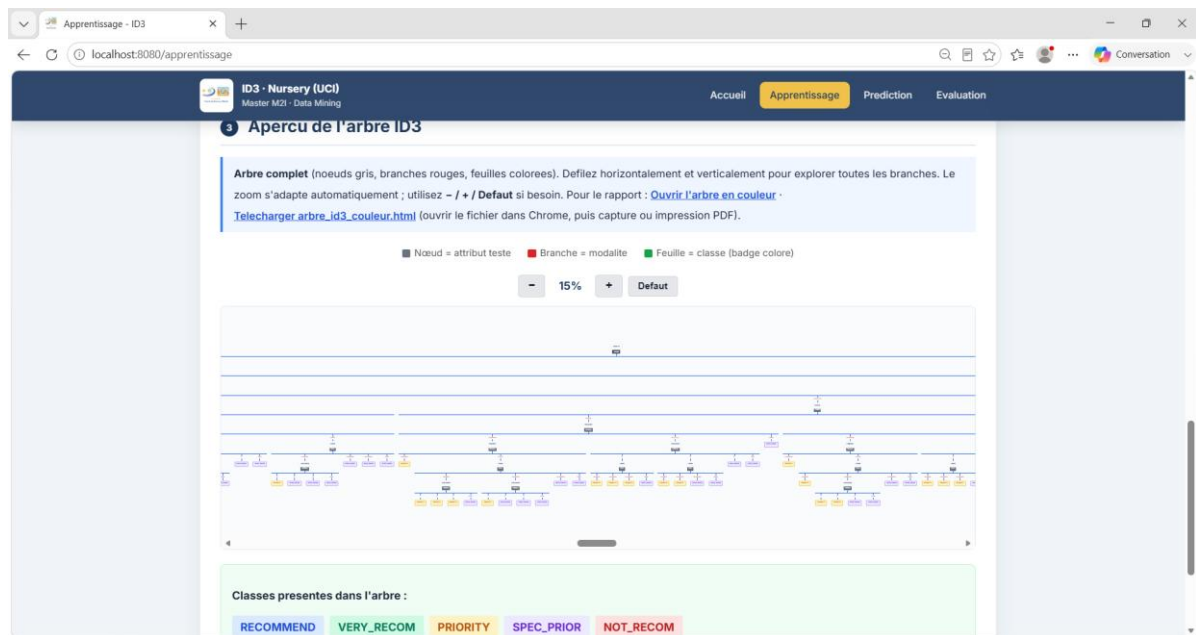


Figure 4.5 — Aperçu interactif de l'arbre ID3

4.3 Page Prédiction

Cette page propose un formulaire de huit listes déroulantes, une par attribut du jeu de données (figure 4.6). L'utilisateur choisit ses valeurs et clique sur « Prédire la classe ».

Figure 4.6 — Formulaire de prédiction

L'application retourne la classe prédite avec un récapitulatif des attributs saisis (figure 4.7), puis affiche le chemin de décision suivi dans l'arbre (figure 4.8) : la succession des couples (attribut, modalité choisie) qui a conduit à la feuille. Cette fonctionnalité est l'un des atouts de l'application — elle rend la décision du modèle interprétable en un coup d'œil.

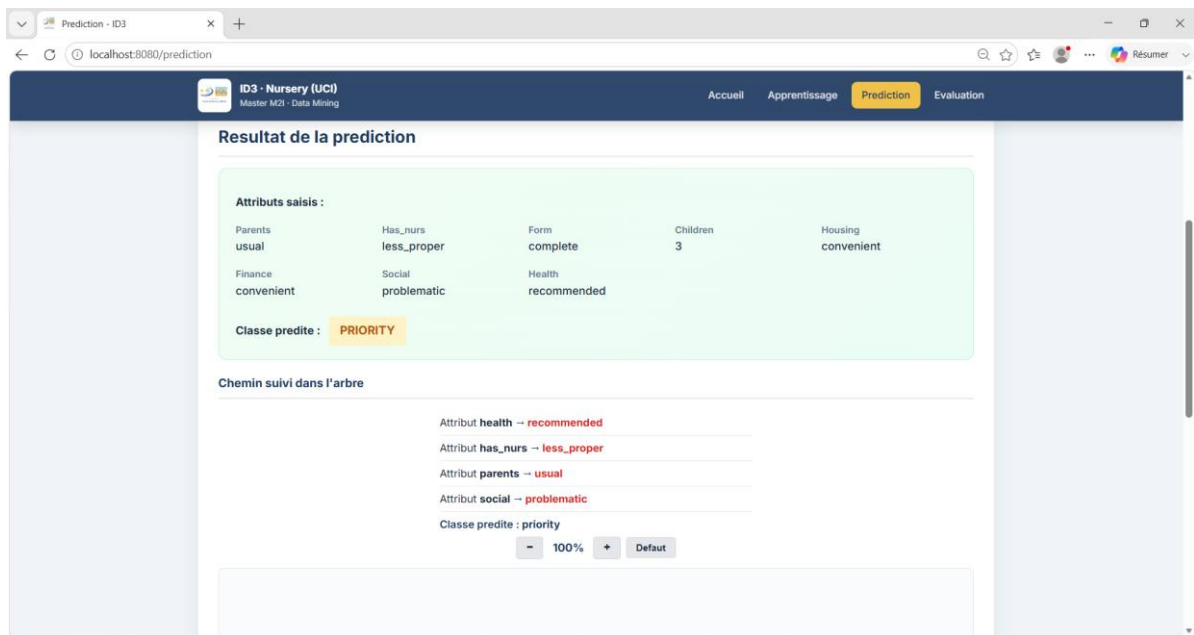


Figure 4.7 — Résultat d'une prédiction

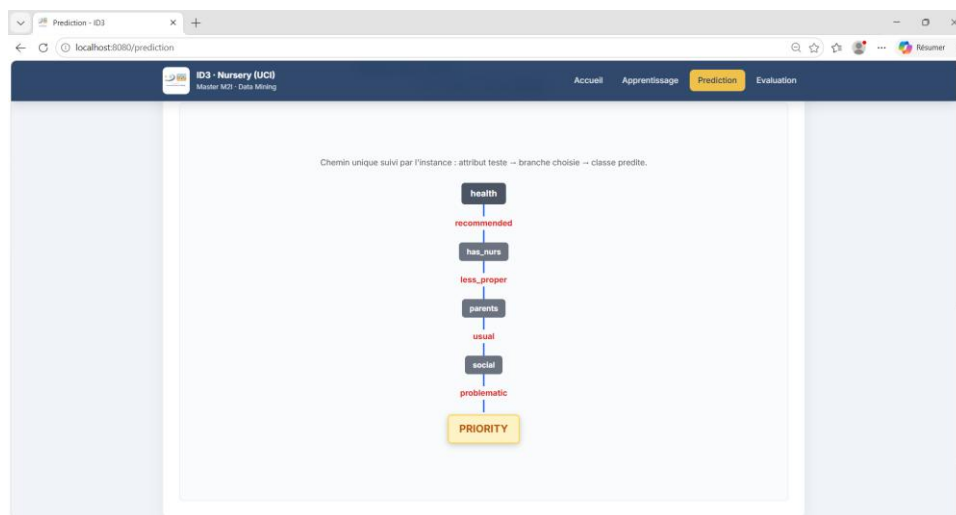


Figure 4.8 — Chemin de décision suivi

4.4 Page Évaluation

La page Évaluation déclenche le calcul des métriques de performance sur le jeu de test (2 592 instances). Elle affiche les quatre indicateurs globaux, le tableau des métriques par classe avec la définition de chaque métrique (figure 4.9), et la matrice de confusion complète (figure 4.10).

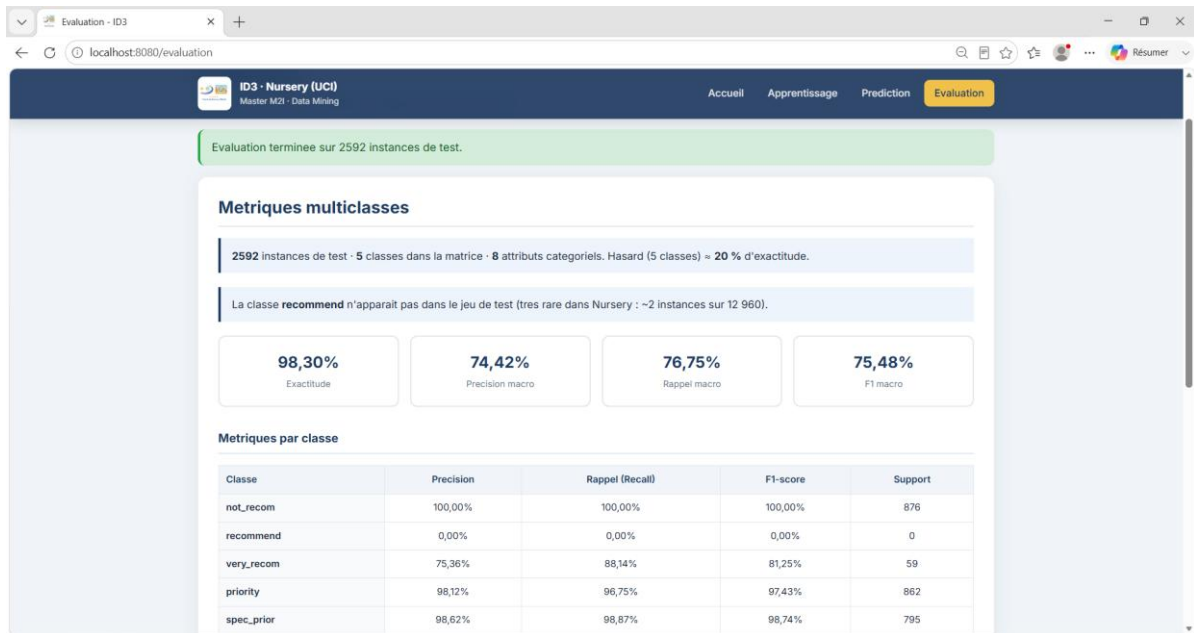


Figure 4.9 — Page Évaluation — métriques globales et par classe

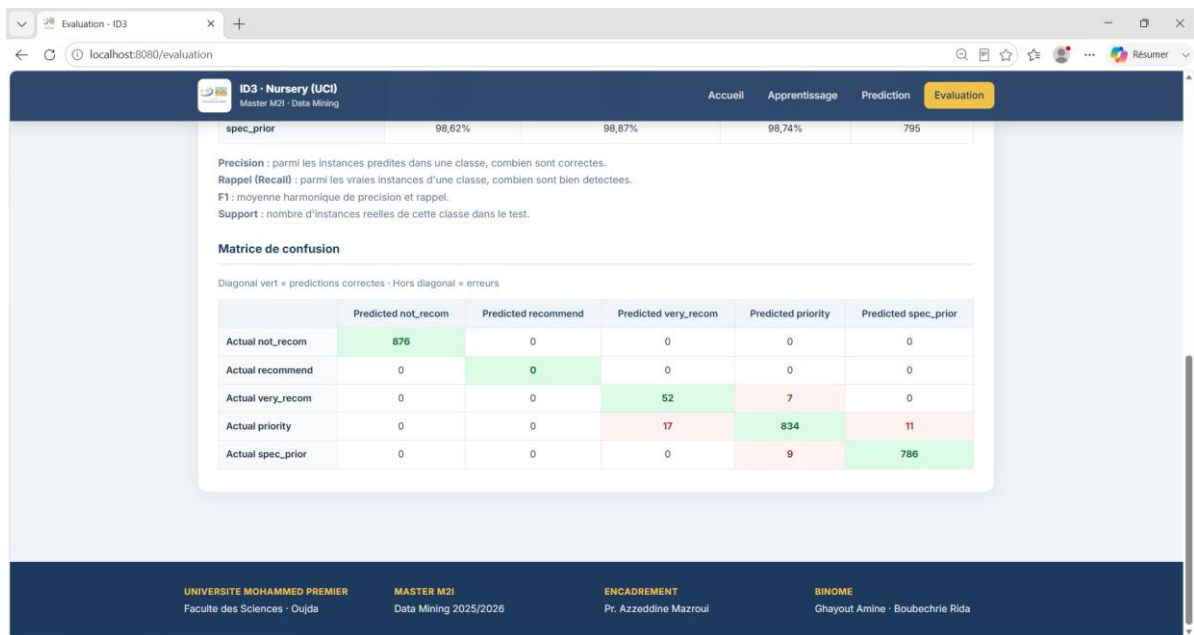
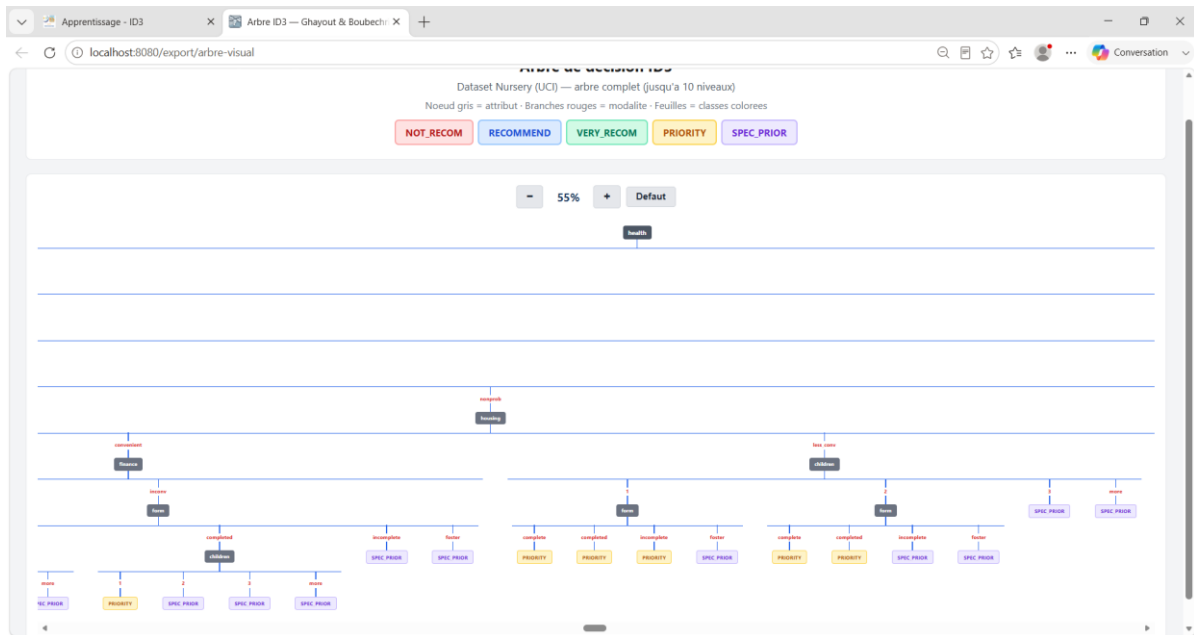


Figure 4.10 — Matrice de confusion

4.5 Export de l'arbre de décision

Depuis la page Apprentissage, un lien permet de télécharger une version HTML autonome et colorisée de l'arbre complet (figure 4.11). Ce fichier s'ouvre dans n'importe quel navigateur sans avoir besoin du serveur Spring Boot.



4.6 Synthèse

L'interface couvre l'intégralité des cas d'utilisation définis au chapitre 2. Chaque page correspond à une étape logique du workflow : charger, apprendre, prédire, évaluer, exporter. Cette organisation rend la démonstration fluide lors de la présentation du projet.

Chapitre 5 — Protocole de test, résultats et discussion

5.1 Protocole expérimental

Le corpus Nursery a été divisé aléatoirement en un ensemble d'apprentissage (80 %) et un ensemble de test (20 %), selon les proportions du tableau 5.1. L'arbre a été construit exclusivement sur les données d'entraînement ; le jeu de test n'intervient qu'au moment de l'évaluation, ce qui garantit une estimation honnête de la capacité de généralisation du modèle.

Ensemble	Instances	Proportion
Entraînement	10 368	80 %
Test	2 592	20 %
Total	12 960	100 %

Tableau 5.1 — Répartition apprentissage / test

Pour chaque instance du jeu de test, la classe prédite par l'arbre est comparée à la classe réelle. On en déduit quatre métriques principales :

- Exactitude (accuracy) : proportion globale d'instances correctement classées.
- Précision par classe : parmi les instances prédites comme appartenant à une classe, quelle fraction y appartient réellement.
- Rappel par classe : parmi les instances qui appartiennent réellement à une classe, quelle fraction a été correctement détectée.
- F1-score : moyenne harmonique de la précision et du rappel.

Les métriques macro correspondent à la moyenne arithmétique, non pondérée par les effectifs, des métriques calculées classe par classe. Elles sont plus informatives que la seule exactitude globale quand les classes sont déséquilibrées.

5.2 Résultats globaux

Indicateur	Valeur
Instances de test	2 592
Exactitude (accuracy)	98,30 %
Précision macro	74,42 %
Rappel macro	76,75 %
F1 macro	75,48 %
Référence hasard (5 classes)	≈ 20 %

Tableau 5.2 — Résultats globaux sur le jeu de test

L'exactitude globale de 98,30 % est très largement au-dessus du hasard (20 % pour cinq classes). L'écart entre ce chiffre et les métriques macro (autour de 75 %) s'explique par le déséquilibre des classes — un point abordé en détail dans la discussion.

5.3 Résultats détaillés par classe

Classe	Précision	Rappel	F1-score	Support
not_recom	100,00 %	100,00 %	100,00 %	876
recommend	0,00 %	0,00 %	0,00 %	0
very_recom	75,36 %	88,14 %	81,25 %	59
priority	98,12 %	96,75 %	97,43 %	862
spec_prior	98,62 %	98,87 %	98,74 %	795

Tableau 5.3 — Métriques de performance par classe

Réelle \ Prédite	not_recom	recommend	very_recom	priority	spec_prior
not_recom	876	0	0	0	0
recommend	0	0	0	0	0
very_recom	0	0	52	7	0
priority	0	0	17	834	11
spec_prior	0	0	0	9	786

Tableau 5.4 — Matrice de confusion — jeu de test

5.4 Discussion

Les résultats appellent plusieurs commentaires selon les classes.

Pour not_recom, priority et spec_prior — qui représentent ensemble plus de 97 % du corpus — les performances sont excellentes. Not_recom atteint un F1 parfait (100 %), priority et spec_prior dépassent tous les deux les 97 %. Ce résultat s'explique par la nature même du dataset Nursery : il a été généré à partir d'un modèle hiérarchique de décision explicite, pas observé sur le terrain. La classe réelle d'une candidature est donc une fonction quasi déterministe de ses attributs, que l'arbre ID3 retrouve presque exactement.

La classe very_recom, minoritaire (328 instances, 2,5 %), obtient un F1 de 81,25 %. Les erreurs se font principalement par confusion avec priority — 7 instances very_recom prédites priority, et 17 instances priority prédites very_recom. Cette confusion entre classes voisines dans la hiérarchie du dataset est cohérente avec la structure du problème.

La classe recommend ne peut pas être évaluée : avec seulement 2 instances sur 12 960, le découpage aléatoire 80/20 ne lui laisse statistiquement aucune instance dans le jeu de test. Précision, rappel et F1 sont donc nuls, non pas parce que le modèle se trompe, mais parce

que la classe n'apparaît pas dans le test. C'est une limite inhérente au dataset, pas à l'algorithme.

C'est cette classe quasi absente qui explique l'écart entre l'exactitude globale (98,30 %) et les métriques macro (~75 %) : la moyenne non pondérée des F1 inclut le zéro de recommend avec autant de poids que les autres classes.

5.5 Synthèse

Le modèle ID3 obtenu atteint une exactitude de 98,30 % sur le jeu de test, avec des performances très satisfaisantes sur quatre des cinq classes. La seule limite identifiée — l'impossibilité d'évaluer la classe recommend — est directement liée au déséquilibre extrême du dataset et ne remet pas en cause la qualité de l'implémentation.

Conclusion générale

Synthèse du travail réalisé

Ce mini-projet nous a permis de mettre en pratique, de bout en bout, les notions du chapitre 5 du cours de Data Mining. Nous sommes partis d'un jeu de données réel (Nursery, UCI) et avons développé, en Java, un moteur ID3 complet — calcul de l'entropie, du gain d'information, construction récursive de l'arbre — sans aucune bibliothèque externe. L'ensemble a été intégré dans une application web Spring Boot dotée d'une interface graphique couvrant toutes les étapes : chargement des données, apprentissage, prédiction interactive avec affichage du chemin de décision, évaluation des performances et export de l'arbre.

Le modèle obtenu atteint une exactitude de 98,30 % sur le jeu de test, ce qui valide l'implémentation réalisée. La seule limite notable — le fait que la classe recommand ne peut pas être apprise ni évaluée faute d'exemples suffisants — est un problème du dataset lui-même.

Difficultés rencontrées

La principale difficulté a été de choisir un dataset réellement adapté au sujet « chaînes de caractères » : il fallait s'assurer que l'ensemble des attributs, sans exception, soient des variables qualitatives textuelles. Le déséquilibre prononcé des classes de Nursery a également nécessité une attention particulière lors de l'interprétation des métriques d'évaluation.

Perspectives d'amélioration

Plusieurs améliorations seraient envisageables pour la suite :

- Mettre en place une validation croisée stratifiée pour s'assurer que chaque classe, même minoritaire, est représentée dans les plis de test.
- Implémenter l'élagage de l'arbre (pruning) pour réduire le sur-apprentissage sur des données plus bruitées que Nursery.
- Comparer ID3 à C4.5 [7] (ratio de gain, gestion des valeurs continues et manquantes) pour mesurer l'apport de ces extensions sur ce corpus.
- Généraliser l'application à n'importe quel fichier CSV catégoriel, et pas uniquement au dataset Nursery.

Bibliographie

- [1] J. R. Quinlan, « Induction of Decision Trees », *Mach. Learn.*, vol. 1, n° 1, p. 81-106, 1986, doi: 10.1007/BF00116251.
- [2] « Spring Boot Reference Documentation ». [En ligne]. Disponible sur: <https://docs.spring.io/spring-boot/docs/3.2.x/reference/html>
- [3] A. Mazroui, « Data Mining — Chapitre 5 : Classification par arbres de décision », 2025.
- [4] M. Olave, V. Rajkovic, et M. Bohanec, « Nursery Data Set ». UCI Machine Learning Repository, University of California, Irvine, 1989. [En ligne]. Disponible sur: <https://archive.ics.uci.edu/dataset/76/nursery>
- [5] T. M. Cover et J. A. Thomas, *Elements of Information Theory*, 2nd éd. Hoboken: John Wiley & Sons, 2006.
- [6] « Mermaid Documentation ». [En ligne]. Disponible sur: <https://mermaid.js.org>
- [7] J. R. Quinlan, *C4.5: Programs for Machine Learning*. San Francisco: Morgan Kaufmann, 1993.